



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e  
INTERACCIÓN HUMANO COMPUTADORA



## **REPORTE DE PRÁCTICA N° 06**

**NOMBRE COMPLETO:** ALFONSO D'HERNÁN RODRÍGUEZ  
ZULUAGA

**N° de Cuenta:** 321561556

**GRUPO DE LABORATORIO:** 02

**GRUPO DE TEORÍA:** 05

**SEMESTRE** 2026-2

**FECHA DE ENTREGA LÍMITE:** 12 de abril del 2026

**CALIFICACIÓN:** \_\_\_\_\_

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1. Dado de 8 lados

Código implementado:

\*Código generado por IA, lo único modificado por mí fueron las coordenadas S, T pertinentes.\*

```
void CrearPiramide_UP()
{
    unsigned int piramide_indices[] = {
        // Base (2 triangles)
        0, 1, 2,
        2, 3, 0,

        // Front face
        4, 5, 6,
        // Right face
        7, 8, 9,
        // Back face
        10, 11, 12,
        // Left face
        13, 14, 15
    };

    GLfloat piramide_vertices[] = {
        // x      y      z      S      T      NX      NY      NZ

        // --- BASE (Bottom) --- Normal: [0, -1, 0]
        -0.5f, -0.5f,  0.5f,  0.0f,  0.0f,  0.0f, -1.0f,  0.0f, // 0
         0.5f, -0.5f,  0.5f,  1.0f,  0.0f,  0.0f, -1.0f,  0.0f, // 1
         0.5f, -0.5f, -0.5f,  1.0f,  1.0f,  0.0f, -1.0f,  0.0f, // 2
        -0.5f, -0.5f, -0.5f,  0.0f,  1.0f,  0.0f, -1.0f,  0.0f, // 3
```

```

// --- FRONT FACE --- Normal: [0, 0.5, 0.8] approx
-0.5f, -0.5f, 0.5f, 0.021f, 0.6511f, 0.0f, 0.44f, 0.89f, // 4
0.5f, -0.5f, 0.5f, 0.21f, 0.6511f, 0.0f, 0.44f, 0.89f, // 5
0.0f, 0.5f, 0.0f, 0.13f, 0.988f, 0.0f, 0.44f, 0.89f, // 6 (Apex)

// --- RIGHT FACE --- Normal: [0.8, 0.5, 0] approx
0.5f, -0.5f, 0.5f, 0.267f, 0.6511f, 0.89f, 0.44f, 0.0f, // 7
0.5f, -0.5f, -0.5f, 0.464f, 0.6511f, 0.89f, 0.44f, 0.0f, // 8
0.0f, 0.5f, 0.0f, 0.36f, 0.988f, 0.89f, 0.44f, 0.0f, // 9 (Apex)

// --- BACK FACE --- Normal: [0, 0.5, -0.8] approx
0.5f, -0.5f, -0.5f, 0.519f, 0.6511f, 0.0f, 0.44f, -0.89f, // 10
-0.5f, -0.5f, -0.5f, 0.71f, 0.6511f, 0.0f, 0.44f, -0.89f, // 11
0.0f, 0.5f, 0.0f, 0.606f, 0.988f, 0.0f, 0.44f, -0.89f, // 12 (Apex)

// --- LEFT FACE --- Normal: [-0.8, 0.5, 0] approx
-0.5f, -0.5f, -0.5f, 0.776f, 0.6511f, -0.89f, 0.44f, 0.0f, // 13
-0.5f, -0.5f, 0.5f, 0.945f, 0.6511f, -0.89f, 0.44f, 0.0f, // 14
0.0f, 0.5f, 0.0f, 0.841f, 0.988f, -0.89f, 0.44f, 0.0f, // 15 (Apex)
};

Mesh* piramide = new Mesh();
// 16 vertices * 8 floats per vertex = 128
// 18 indices (6 for base, 3 per side * 4 sides)
piramide->CreateMesh(piramide_vertices, piramide_indices, 128, 18);
meshList.push_back(piramide);

```

\*Esta función fue una modificación de la primera, por lo que la sigo contando como código hecho por IA\*

```

void CrearPiramide_DOWN()
{
    unsigned int piramide_indices[] = {
        // Base (2 triangles)
        0, 1, 2,
        2, 3, 0,

        // Front face
        4, 5, 6,
        // Right face
        7, 8, 9,
        // Back face
        10, 11, 12,
        // Left face
        13, 14, 15
    };

    GLfloat piramide_vertices[] = {
        // x      y      z      S      T      NX      NY      NZ

        // --- BASE (Bottom) --- Normal: [0, -1, 0]
        -0.5f, -0.5f, 0.5f, 0.0f, 0.0f, 0.0f, -1.0f, 0.0f, // 0
        0.5f, -0.5f, 0.5f, 1.0f, 0.0f, 0.0f, -1.0f, 0.0f, // 1
        0.5f, -0.5f, -0.5f, 1.0f, 1.0f, 0.0f, -1.0f, 0.0f, // 2
        -0.5f, -0.5f, -0.5f, 0.0f, 1.0f, 0.0f, -1.0f, 0.0f, // 3

        // --- FRONT FACE --- Normal: [0, 0.5, 0.8] approx
        -0.5f, -0.5f, 0.5f, 0.0f, 0.0465f, 0.0f, 0.44f, 0.89f, // 4
        0.5f, -0.5f, 0.5f, 0.196f, 0.0465f, 0.0f, 0.44f, 0.89f, // 5
        0.0f, 0.5f, 0.0f, 0.103f, 0.384f, 0.0f, 0.44f, 0.89f, // 6 (Apex)
    };
}

```

```

// --- RIGHT FACE --- Normal: [0.8, 0.5, 0] approx
0.5f, -0.5f, 0.5f, 0.267f, 0.0465f, 0.89f, 0.44f, 0.0f, // 7
0.5f, -0.5f, -0.5f, 0.464f, 0.0465f, 0.89f, 0.44f, 0.0f, // 8
0.0f, 0.5f, 0.0f, 0.36f, 0.384f, 0.89f, 0.44f, 0.0f, // 9 (Apex)

// --- BACK FACE --- Normal: [0, 0.5, -0.8] approx
0.5f, -0.5f, -0.5f, 0.540f, 0.0232f, 0.0f, 0.44f, -0.89f, // 10
-0.5f, -0.5f, -0.5f, 0.743f, 0.0465f, 0.0f, 0.44f, -0.89f, // 11
0.0f, 0.5f, 0.0f, 0.639f, 0.372f, 0.0f, 0.44f, -0.89f, // 12 (Apex)

// --- LEFT FACE --- Normal: [-0.8, 0.5, 0] approx
-0.5f, -0.5f, -0.5f, 0.792f, 0.0581f, -0.89f, 0.44f, 0.0f, // 13
-0.5f, -0.5f, 0.5f, 0.989f, 0.093f, -0.89f, 0.44f, 0.0f, // 14
0.0f, 0.5f, 0.0f, 0.885f, 0.441f, -0.89f, 0.44f, 0.0f, // 15 (Apex)
};

Mesh* piramide = new Mesh();
// 16 vertices * 8 floats per vertex = 128
// 18 indices (6 for base, 3 per side * 4 sides)
piramide->CreateMesh(piramide_vertices, piramide_indices, 128, 18);
meshList.push_back(piramide);

```

\*Aquí acaba código generado por IA\*

```

model = glm::mat4(1.0);

model = glm::translate(model, glm::vec3(-0.0f, 20.0f, 3.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(4.0f, 4.00f, 4.00f));

glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
dice_eight.UseTexture();
meshList[5]->RenderMesh();

model = modelaux;
model = glm::translate(model, glm::vec3(-0.0f, -4.0f, 0.0f));
model = glm::scale(model, glm::vec3(4.0f, 4.00f, 4.00f));
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
dice_eight.UseTexture();
meshList[6]->RenderMesh();

glUseProgram(0);

mainWindow.swapBuffers();

```

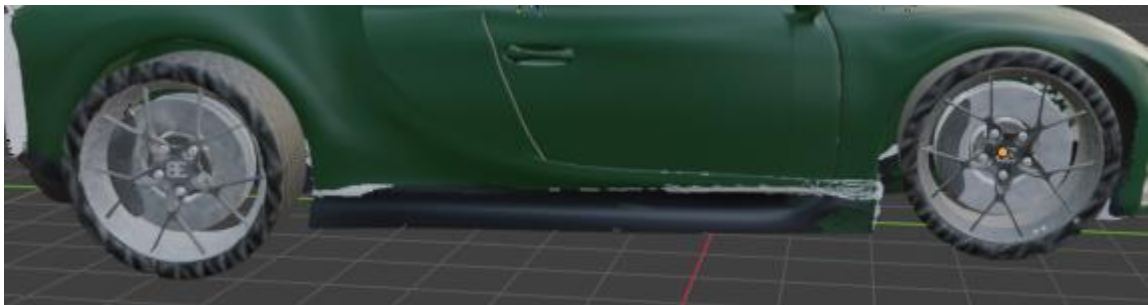
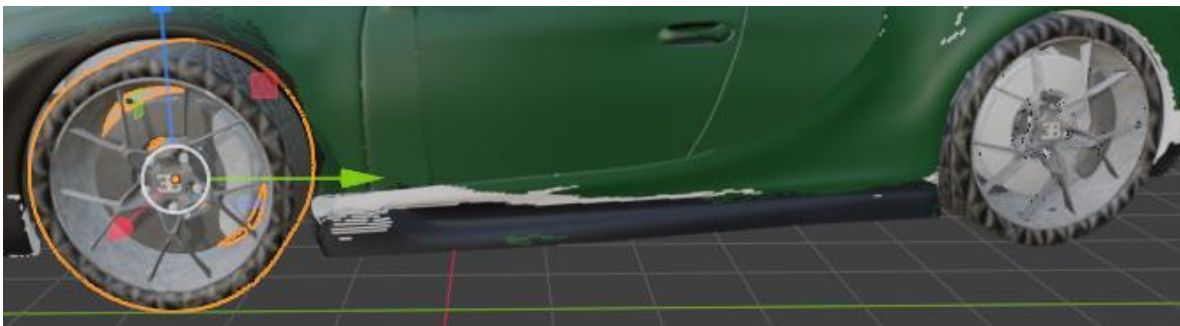
Visualización de los resultados:





## 2. Llantas texturizadas

Llantas en Blender:



Código de las llantas:



```

model = glm::translate(model, glm::vec3(-0.3f, 0.3f, -0.4f));
model = glm::rotate(model, glm::radians(10.0f), glm::vec3(0.0f, 1.0f, 0.0f));
//model = glm::scale(model, glm::vec3(30.0f, 30.0f, 30.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Carro_PDD.RenderModel();
model = modelaux;

model = glm::translate(model, glm::vec3(1.3f, -0.75f, 2.1f));
model = glm::rotate(model, glm::radians(10.0f), glm::vec3(0.0f, 1.0f, 0.0f));
//model = glm::scale(model, glm::vec3(30.0f, 30.0f, 30.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Carro_PDI.RenderModel();
model = modelaux;

model = glm::translate(model, glm::vec3(1.0f, 0.0f, -1.2f));
model = glm::rotate(model, glm::radians(10.0f), glm::vec3(0.0f, 1.0f, 0.0f));
//model = glm::scale(model, glm::vec3(30.0f, 30.0f, 30.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Carro_PTD.RenderModel();
model = modelaux;

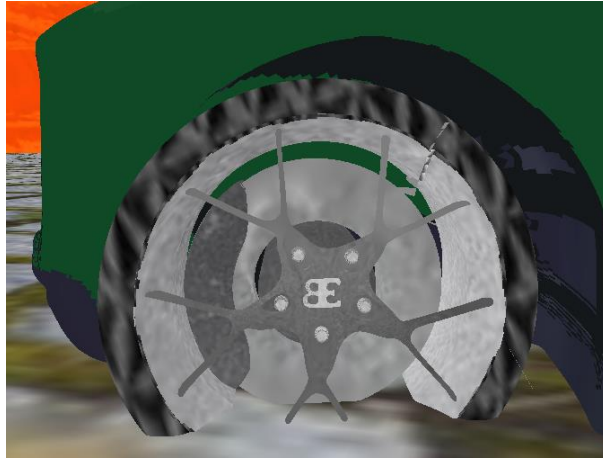
model = glm::translate(model, glm::vec3(0.8f, 0.25f, -0.0f));
model = glm::rotate(model, glm::radians(10.0f), glm::vec3(0.0f, 1.0f, 0.0f));

//model = glm::scale(model, glm::vec3(30.0f, 30.0f, 30.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Carro_PTI.RenderModel();
model = modelaux;

```

Visualización de las llantas:

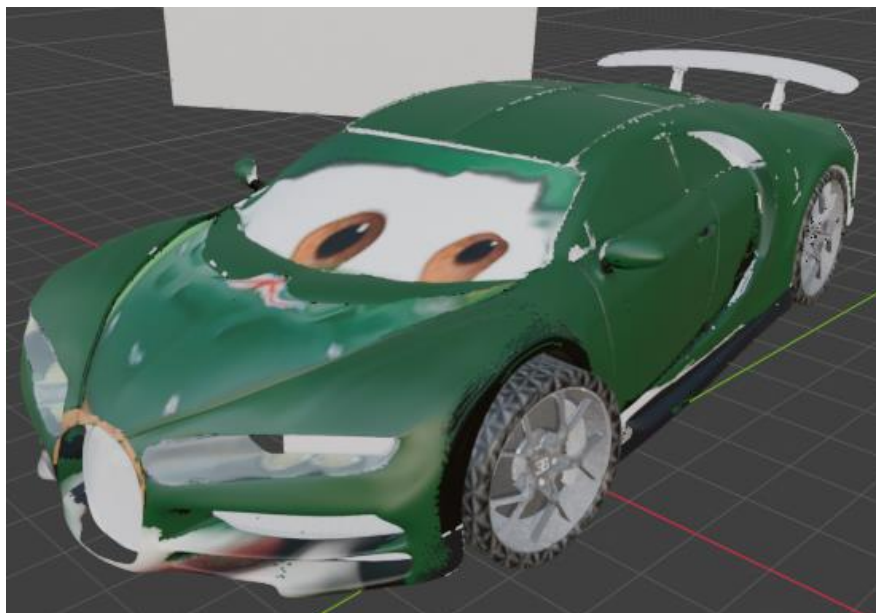




### 3. Vehículo texturizado

Vehículo en blender:





Código del vehículo

```

//Instancia del coche
model = glm::mat4(1.0);
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
model = glm::translate(model, glm::vec3(0.0f, -2.0f, -7.5f));
modelaux = model;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Carro_M.RenderModel();
model = modelaux;

model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
//model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
//model = glm::rotate(model, glm::radians(-10.0f), glm::vec3(0.0f, 0.0f, 1.0f));
//model = glm::scale(model, glm::vec3(30.0f, 30.0f, 30.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Carro_Cofre.RenderModel();

```

Vehículo visualizado:



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

- Tuve problemas para que se me ocurriera cómo implementar la pirámide cuadrangular con coordenadas S y T, esto debido a que sí tenía el código de las prácticas pasadas y sí tenía la idea de cómo se podía implementar, pero empecé a tener problemas de tipado y de mediciones, por lo que terminó siendo una pérdida de tiempo. Considerando lo anterior, finalmente le pedí a la inteligencia artificial que me ayudara con ello. La conversación fue la siguiente:

PROMPT:

```
void CrearDado()
```

```
{
```

```
    unsigned int cubo_indices[] = {
```

```
        // front
```

```
        0, 1, 2,
```

```
        2, 3, 0,
```

```
        // back
```

```
        8, 9, 10,
```

```
        10, 11, 8,
```

```
        // left
```

```
        12, 13, 14,
```

```
        14, 15, 12,
```

```
        // bottom
```

```
        16, 17, 18,
```

```
        18, 19, 16,
```

```
        // top
```

```
        20, 21, 22,
```

```
        22, 23, 20,
```

```
        // right
```

```
        4, 5, 6,
```

```
        6, 7, 4,
```

```
    };
```

//Ejercicio 1: reemplazar con sus datos de 6 caras texturizados, agregar normales

// average normals

GLfloat cubo\_vertices[] = {

// front

//x y z S T NX NY NZ

-0.5f, -0.5f, 0.5f, 0.35f, 0.51f, 0.0f, 0.0f, -1.0f, //0

0.5f, -0.5f, 0.5f, 0.645f, 0.51f, 0.0f, 0.0f, -1.0f, //1

0.5f, 0.5f, 0.5f, 0.645f, 0.74f, 0.0f, 0.0f, -1.0f, //2

-0.5f, 0.5f, 0.5f, 0.35f, 0.74f, 0.0f, 0.0f, -1.0f, //3

// right

//x y z S T

0.5f, -0.5f, 0.5f, 0.67f, 0.53f, -1.0f, 0.0f, 0.0f,

0.5f, -0.5f, -0.5f, 0.965f, 0.53f, -1.0f, 0.0f, 0.0f,

0.5f, 0.5f, -0.5f, 0.965f, 0.75f, -1.0f, 0.0f, 0.0f,

0.5f, 0.5f, 0.5f, 0.67f, 0.75f, -1.0f, 0.0f, 0.0f,

// back

-0.5f, -0.5f, -0.5f, 0.34f, 0.24f, 0.0f, 0.0f, 1.0f,

0.5f, -0.5f, -0.5f, 0.645f, 0.24f, 0.0f, 0.0f, 1.0f,

0.5f, 0.5f, -0.5f, 0.645f, 0.01f, 0.0f, 0.0f, 1.0f,

-0.5f, 0.5f, -0.5f, 0.34f, 0.01f, 0.0f, 0.0f, 1.0f,

// left

//x y z S T

-0.5f, -0.5f, -0.5f, 0.01f, 0.51f, 1.0f, 0.0f, 0.0f,

-0.5f, -0.5f, 0.5f, 0.32f, 0.51f, 1.0f, 0.0f, 0.0f,

-0.5f, 0.5f, 0.5f, 0.32f, 0.74f, 1.0f, 0.0f, 0.0f,

-0.5f, 0.5f, -0.5f, 0.01f, 0.74f, 1.0f, 0.0f, 0.0f,

// bottom

//x y z S T

-0.5f, -0.5f, 0.5f, 0.34f, 0.5f, 0.0f, 1.0f, 0.0f,

0.5f, -0.5f, 0.5f, 0.65f, 0.5f, 0.0f, 1.0f, 0.0f,

0.5f, -0.5f, -0.5f, 0.65f, 0.26f, 0.0f, 1.0f, 0.0f,

```
-0.5f, -0.5f, -0.5f, 0.34f, 0.26f, 0.0f, 1.0f, 0.0f,
```

```
//UP
```

```
//x y z S T
```

```
-0.5f, 0.5f, 0.5f, 0.35f, 0.77f, 0.0f, -1.0f, 0.0f,
```

```
0.5f, 0.5f, 0.5f, 0.655f, 0.77f, 0.0f, -1.0f, 0.0f,
```

```
0.5f, 0.5f, -0.5f, 0.655f, 0.99f, 0.0f, -1.0f, 0.0f,
```

```
-0.5f, 0.5f, -0.5f, 0.35f, 0.99f, 0.0f, -1.0f, 0.0f,
```

```
};
```

```
Mesh* dado = new Mesh();
```

```
dado->CreateMesh(cubo_vertices, cubo_indices, 192, 36);
```

```
meshList.push_back(dado);
```

```
}
```

I need you to adapt this OpenGL code in order for me to be able to render a pyramid along with a TGA texture.

RESPUESTA:

To adapt your code for a pyramid, we need to redefine the vertices (positions, texture coordinates, and normals) and the indices (how the triangles are connected).

A pyramid typically has 5 vertices (4 for the base and 1 for the peak), but for correct texturing and lighting (normals), we must duplicate vertices so each face has its own set. This prevents the texture and lighting from "stretching" or averaging incorrectly across edges.

Pyramid Specifications

- Base: 4 vertices (1 square made of 2 triangles).
- Sides: 4 triangles (3 vertices each).
- Normals: Pointing outwards from each face for correct lighting.

---

Adapted Code: CrearPiramide()

C++

```
void CrearPiramide()
```

```
{
```

```

unsigned int piramide_indices[] = {
    // Base (2 triangles)
    0, 1, 2,
    2, 3, 0,

    // Front face
    4, 5, 6,
    // Right face
    7, 8, 9,
    // Back face
    10, 11, 12,
    // Left face
    13, 14, 15
};

```

```

GLfloat piramide_vertices[] = {
    // x    y    z    S    T    NX    NY    NZ

    // --- BASE (Bottom) --- Normal: [0, -1, 0]
    -0.5f, -0.5f, 0.5f, 0.0f, 0.0f, 0.0f, -1.0f, 0.0f, // 0
    0.5f, -0.5f, 0.5f, 1.0f, 0.0f, 0.0f, -1.0f, 0.0f, // 1
    0.5f, -0.5f, -0.5f, 1.0f, 1.0f, 0.0f, -1.0f, 0.0f, // 2
    -0.5f, -0.5f, -0.5f, 0.0f, 1.0f, 0.0f, -1.0f, 0.0f, // 3

    // --- FRONT FACE --- Normal: [0, 0.5, 0.8] approx
    -0.5f, -0.5f, 0.5f, 0.0f, 0.0f, 0.0f, 0.44f, 0.89f, // 4
    0.5f, -0.5f, 0.5f, 1.0f, 0.0f, 0.0f, 0.44f, 0.89f, // 5
    0.0f, 0.5f, 0.0f, 0.5f, 1.0f, 0.0f, 0.44f, 0.89f, // 6 (Apex)

    // --- RIGHT FACE --- Normal: [0.8, 0.5, 0] approx

```



```

0.5f, -0.5f, 0.5f, 0.0f, 0.0f, 0.89f, 0.44f, 0.0f, // 7
0.5f, -0.5f, -0.5f, 1.0f, 0.0f, 0.89f, 0.44f, 0.0f, // 8
0.0f, 0.5f, 0.0f, 0.5f, 1.0f, 0.89f, 0.44f, 0.0f, // 9 (Apex)

// --- BACK FACE --- Normal: [0, 0.5, -0.8] approx
0.5f, -0.5f, -0.5f, 0.0f, 0.0f, 0.0f, 0.44f, -0.89f, // 10
-0.5f, -0.5f, -0.5f, 1.0f, 0.0f, 0.0f, 0.44f, -0.89f, // 11
0.0f, 0.5f, 0.0f, 0.5f, 1.0f, 0.0f, 0.44f, -0.89f, // 12 (Apex)

// --- LEFT FACE --- Normal: [-0.8, 0.5, 0] approx
-0.5f, -0.5f, -0.5f, 0.0f, 0.0f, -0.89f, 0.44f, 0.0f, // 13
-0.5f, -0.5f, 0.5f, 1.0f, 0.0f, -0.89f, 0.44f, 0.0f, // 14
0.0f, 0.5f, 0.0f, 0.5f, 1.0f, -0.89f, 0.44f, 0.0f // 15 (Apex)
};

Mesh* piramide = new Mesh();
// 16 vertices * 8 floats per vertex = 128
// 18 indices (6 for base, 3 per side * 4 sides)
piramide->CreateMesh(piramide_vertices, piramide_indices, 128, 18);
meshList.push_back(piramide);
}

```

### **PROMPT:**

Where did you get this information? Please cite it.

### **RESPUESTA:**

As an AI, I synthesize information from established computer graphics principles and technical documentation rather than a single source. The code I provided is based on the Vertex Array Object (VAO) and Index Buffer standards used in modern OpenGL (3.3+).

Specifically, the structure and logic were derived from the following sources and methods:

#### 1. Geometry and Texture Mapping

- Source: [LearnOpenGL.com - Textures](https://learnopengl.com/Textures)

- Logic: For the triangular faces, I used the standard mapping where the base of the triangle is at  $(0,0)$  coordinates and the apex is at  $(0.5, 1.0)$ . This ensures the image is centered on each face of the pyramid.

## 2. Normal Vector Calculations

- Source: [Megabyte Softworks - Normals and Diffuse Lighting](#)
- Logic: Lighting requires "Normal" vectors. I calculated these using the Cross Product of the edges  $(\vec{B} - \vec{A}) \times (\vec{C} - \vec{A})$ . For a square pyramid with height 1.0 and width 1.0, the normalized vectors for the side faces are approximately  $(\pm 0.89, 0.44, 0.0)$  or  $(0.0, 0.44, \pm 0.89)$ .

## 3. Vertex Duplication (Smoothing vs. Flat Shading)

- Source: [SongHo.ca - OpenGL Cone & Pyramid](#)
- Logic: To prevent "bleeding" of textures and lighting artifacts, I followed the "Flat Shading" approach. This requires duplicating the apex vertex four times. Even though it is the same point in 3D space, each face needs a unique Normal and Texture coordinate for that point.

## 4. TGA Loading and Texture Units

- Source: [OGLdev - Basic Texture Mapping](#)
- Logic: The integration with TGA files assumes a standard sampler2D in GLSL, which is the industry standard for wrapping images (TGA, PNG, JPG) onto geometry in OpenGL.

### ¿Cómo me cercioré de que este código funcionase?

Cuando empecé a hacer uso del código en OpenGL y empecé a adaptar el mismo a mi imagen de tipo TGA, empecé a ver que los vértices sí se adaptaban a las partes de la imagen donde yo deseaba que estuvieran, por lo que la comprobación fue empírica.

### ¿Cuál fue el alcance de este código?

El alcance de este código fue el ejercicio 1, pues me apoyé de este para renderizar la pirámide con toda la información necesaria. Sin embargo, este código no aplicó para la sección 2 de la práctica, pues esta ya fue enteramente de autoría mía.

- También tuve un poco de problema para medir las ubicaciones de las vértices en la imagen del dado, pero para resolverlo hice uso de una calculadora y utilicé variables para ir cambiando los valores y adaptarlos a S, T, de manera que pude adjuntar los valores necesarios.

- Tuve problemas bastante grandes con el texturizado del vehículo en particular, pues el UV mapping y demás lo tenía bien hecho. Sin embargo, cuando puse la primera imagen utilizada, simplemente no estaba funcionando, tenía muchos colores sin sentido aparente y no se veía bien el renderizado. No fue hasta que cambié el formato de la imagen de texturizado de PNG a JPG que ya se reflejaron debidamente los resultados.

- También tuve problemas con los UV maps, pensé que se debían de hacer en UV maps separados, hasta que después de jugar un poco con las opciones y ver que los UV maps no funcionaban, vi que se podía hacer un UV map por cada región y en el mismo UV editing, asignar texturas diferentes. También me apoyé de foros en línea, donde explicaban cómo remover mapas UV (particularmente la respuesta de Meninthal en [https://www.reddit.com/r/blender/comments/1fjimqa/is\\_there\\_a\\_way\\_to\\_remove\\_a\\_bunch\\_of\\_textures\\_from/](https://www.reddit.com/r/blender/comments/1fjimqa/is_there_a_way_to_remove_a_bunch_of_textures_from/), "If you want -all- textures or materials, then just use the Blender File Display option in the Outliner panel and delete the images or materials directly from there...")

### 3.- Conclusión:

- a. Los ejercicios del reporte estaban en un nivel bastante más alto que el visto en clase, particularmente la parte de los mapas UV. Como en clase solo vimos un mapa UV que era de un cubo, aquél fue muy simple de implementar. Comparado con este, era muy fácil, por lo que este requirió de bastante más tiempo y esfuerzo.
- b. Comentarios generales: Todo estuvo bien a lo largo de la práctica. Si acaso, me siento un poco mal de no haber podido hacerlo mejor pese a mi esfuerzo.

Concluyendo, esta práctica ya se empieza a adentrar en las cuestiones más técnicas de Blender en el sentido de estilizado y de cómo poder dar vida a los modelos que nosotros importamos por medio del texturizado y de la teoría básica de cómo funcionan las texturas. Con esta información a la mano, ya podremos dar vida a los proyectos futuros en los que trabajemos, así logrando que se tenga una animación completa con texturas, así haciéndome pensar que esta práctica está satisfactoriamente concluida.

### Bibliografía en formato APA

Google Gemini. (2026, April 11). *Response regarding OpenGL pyramid geometry and vertex normal calculations* [Large language model]. <https://gemini.google.com/share/e7b65e51a05d>

User. (2024, September 18). Is there a way to remove a bunch of textures from a project? [Online forum post]. Reddit. [https://www.reddit.com/r/blender/comments/1fjimqa/is\\_there\\_a\\_way\\_to\\_remove\\_a\\_bunch\\_of\\_textures\\_from/](https://www.reddit.com/r/blender/comments/1fjimqa/is_there_a_way_to_remove_a_bunch_of_textures_from/)

Stan\_Pancakes. (2015, April 2). How to delete/reset an image map in the UV/image editor? [Online forum post]. Blender Artists. <https://blenderartists.org/t/how-to-delete-reset-an-image-map-in-the-uv-image-editor/636065/2>

